RÉPUBLIQUE FRANÇAISE

Liberté • Égalité • Fraternité

UNIVERSITÉ
CÔTE D'AZUR



COMPTE RENDU DU TP 1: Programmation de FPGA + synthèse

MASTER M1 : ESTEL

Chargé du TP : **Mme Belleudy**

PRESENTE PAR : BALDE ALHASSANE, DIALLO FATOUMATA BINTA I

& AKOHOUVI AMOU REQUEST BERNICE

INTRODUCTION

Ce TP a pour but de nous familiariser avec la programmation de circuits intégrés programmables (FPGA) plus spécifiquement avec la famille Cyclone III. Dans ce TP nous apprendrons à concevoir, simuler et implémenter des circuits numériques sur ce type de matériel**.**

Il est divisé en deux (2) grandes parties qui nous permettront d'explorer les différentes facettes de la programmation FPGA telle que :

-Architecture du Cyclone III: Où nous étudierons l'architecture de base du FPGA Cyclone III, ses éléments constitutifs (cellules logiques, mémoires, etc.) et leur fonctionnement.

-Synthèse FPGA et temps de réponse: Où nous implémenterons différents opérateurs arithmétiques (additionneur, multiplicateur) sur le FPGA et analyserons leur temps de réponse en fonction de différents paramètres de configuration.

-Filtrage vidéo: en fin nous essaierons de développer un filtre vidéo simple et l'implémenter sur le FPGA.

PARTIE 1 : ARCHITECTURE ET CARACTERISTIQUES TECHNIQUES

La famille de FPGA utilisé est Cyclone III. Etudions l'architecture du composant et des cellules élémentaires :

Structure générale du Cyclone III

L'architecture d'un FPGA Cyclone III est constituée d'un réseau organisé de Logic Array Blocks (LABs), qui eux-mêmes contiennent des éléments logiques (LE). Autour de ces blocs logiques se trouvent des ressources supplémentaires comme des blocs mémoires, des blocs DSP, des PLL (Phase-Locked Loops) et des blocs d'entrées/sorties.

1. Logic Array Blocks (LABs)

Chaque FPGA Cyclone III comprend un certain nombre de Logic Array Blocks (LABs). Chaque LAB contient 16 éléments logiques (LEs).

Ces LABs sont placés dans une matrice interconnectée, permettant des connexions logiques et des routages flexibles. Les interconnexions entre les LABs sont gérées via un réseau d'interconnexion hiérarchique qui permet de créer des circuits logiques complexes.

2. Éléments logiques (Logic Elements - LEs)

Les éléments logiques (LEs) sont les unités de base de calcul dans les FPGA. Chacun des LEs contient : Une LUT (Look-Up Table) à 4 entrées : Il s'agit d'une table de correspondance qui permet de stocker et de restituer les résultats d'une fonction logique booléenne de 4 variables, Un registre (flip-flop) et des circuits logiques arithmétiques

3. Réseau d'interconnexion programmables : Les LABs dans les FPGA Cyclone III sont connectés via un réseau de routage programmable qui permet de relier les différentes ressources

logiques (LABs, blocs mémoire, blocs DSP, etc.). Ce réseau comprend des lignes d'interconnexion globales et locales pour acheminer les signaux à travers le FPGA..

4. Blocs de mémoire embarquée (Memory Blocks)

Les FPGA Cyclone III sont dotés de blocs mémoire embarqués, appelés M9K Memory Blocks. Chaque bloc M9K est un segment de 9 Kbits de mémoire RAM, pouvant être utilisé de plusieurs maniéres

Les blocs mémoire sont distribués dans l'architecture et peuvent être reliés aux LABs via le réseau d'interconnexion.

5. Blocs DSP (Digital Signal Processing)

Les FPGA Cyclone III incluent des blocs DSP spécialisés, conçus pour exécuter rapidement des opérations mathématiques, comme Multiplication, Addition et accumulation

Ces blocs DSP permettent d'accélérer les calculs dans des applications en ressources, telles que le traitement d'images, la reconnaissance vocale, les communications sans fil, et le traitement vidéo.

6. Blocs PLL (Phase-Locked Loop)

Les FPGA Cyclone III intègrent plusieurs PLL qui servent à générer des signaux d'horloge avec des fréquences variables et synchronisées.

7. Entrées/Sorties configurables (I/O Blocks)

Les blocs I/O du Cyclone III sont très configurables et supportent de multiples standards d'interfaçage (LVTTL, LVCMOS, SSTL, HSTL, etc.).

8. Configuration SRAM et Reprogrammabilité

Les FPGA Cyclone III sont basés sur une architecture SRAM, ce qui signifie qu'ils sont reprogrammables. La configuration des LUTs, des réseaux d'interconnexion et des blocs mémoire est stockée dans des cellules SRAM. Ces FPGA peuvent être reconfigurés à plusieurs reprises, ce qui les rend particulièrement utiles pour des applications évolutives ou nécessitant des mises à jour de fonctionnalité après la fabrication.

PARTIE 2 : SYNTHESE FPGA ET TEMPS DE REPONSE

Cette 2éme partie vise à analyser les performances et les ressources utilisées pour implémenter les différents opérateurs arithmétiques sur un FPGA Cyclone III.

Notre travail sera d'évaluer l'impact du pipeline sur le temps de réponse et la consommation de ressources d'un additionneur 32 bits, de comparer les résultats obtenus avec différentes configurations de pipeline (non pipeline, 2 étages, 4 étages), d'analyser les spécificités de l'addition flottante au niveau de l'implémentation FPGA, d'étudier l'impact des différentes configurations sur le mapping et le temps de réponse, de synthétiser un filtre vidéo simple basé sur une convolution et d'évaluer les performances du filtre en termes de temps de traitement.

2.1- Dans cette partie le but est d'étudier l'évolution du temps de réponse et de la complexité d'un additionneur 32 bits non signé en fonction du niveau de pipeline, quantifier l'impact du pipeline sur les performances et les ressources d'un additionneur 32 bits, en termes de temps de réponse et de nombre de Logique Élémentaires (LE) et en fin nous comparerons à travers un tableau le temps de réponse d'un additionneur non pipeline à des versions pipelinées à 2 et 4 étages.

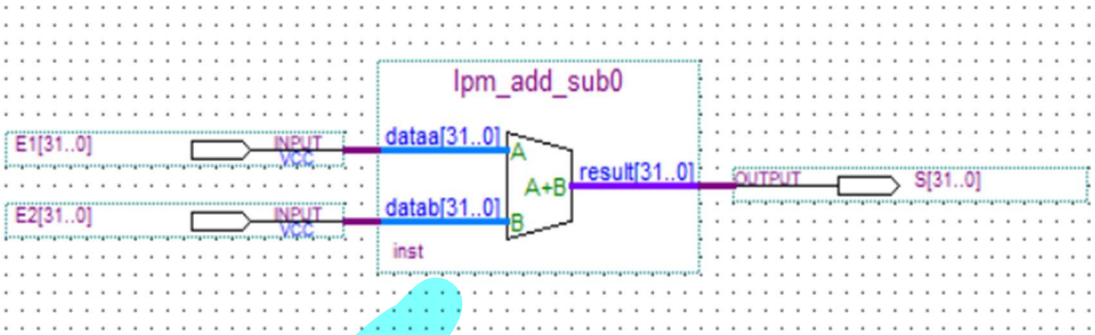


Figure 1: additionneur à 32 bits_ non pipeline

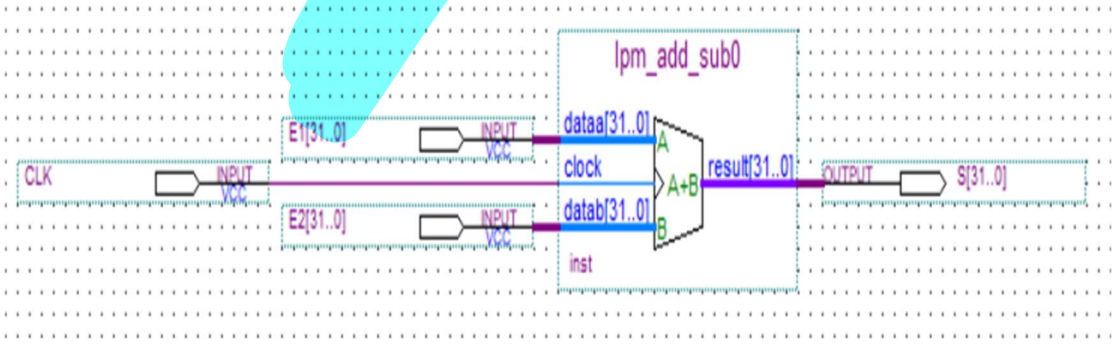


Figure2: additionneur à 32 bits_ avec pipeline

Tableau d'étude des différents temps de réponse

	Non-Pipeline	Pipeline 2	Pipeline 4
Logique élément (LE)	32/15.408	64/15.408	40/15.408
Tsu		4,610 ns	5,100 ns
Tcom		7,142 ns	7,736 ns
Th		0,378 ns	0,191 ns
Tpd	11,844 ns		
Σ(Tcom+Tsu)		11,752 ns	12,836 ns

Analyse et Commentaire :

1. **Logique élément (LE)** : On observe une augmentation du nombre d'éléments logiques utilisés dans la configuration pipeline à 2 étages (64 LE) par rapport à la version non pipeline (32 LE), mais une réduction à 40 LE dans la configuration pipeline à 4 étages. Cela pourrait être dû à des optimisations dans la répartition du calcul sur plusieurs étages.

2. **Temps de setup (Tsu)** :On observe une légère augmentation dans le pipeline à 4 étages par rapport au pipeline 2, cela indique qu'il devient plus complexe de stabiliser les données en ajoutant des étages de pipeline.
3. **Temps de calcul (Tcom)** : On observe aussi une légère augmentation dans le pipeline à 4 étages par rapport aux autres configurations, cela pourrait être dû à la complexité supplémentaire introduite par la gestion des multiples étages de pipeline, qui augmente le délai combinatoire.
4. **Somme (Tcom + Tsu)** : Nous constatons que le pipeline à 4 étages présente un délai total plus élevé (12,836 ns), ce qui montre que l'ajout d'étages supplémentaires n'entraîne pas nécessairement une amélioration des performances dans ce cas.

2.2 - On se propose maintenant d'étudier un additionneur sur les flottants.

Rappelons comment s'effectue cette opération : l'addition de nombres flottants est une opération plus complexe que l'addition d'entiers en raison de leur représentation spécifique : un signe, une mantisse et un exposant.

Les étapes principales de l'addition de deux nombres flottants sont les suivantes :

1. Alignement des mantisse: il s'agit d'identifier l'exposant le plus grand, le nombre avec l'exposant le plus petit est décalé vers la droite jusqu'à ce que les deux exposants soient égaux. Ajuster la mantisse: à chaque décalage vers la droite, le poids des bits de la mantisse est divisé par 2. Cela revient à ajuster l'exposant du nombre décalé.
2. Addition des mantisses: les mantisses alignées sont additionnées comme des nombres entiers.
3. Normalisation: Si le résultat de l'addition a un dépassement de capacité ou un zéro non significatif en tête , il faut normaliser le résultat. Cela consiste à décaler la mantisse vers la gauche ou la droite et à ajuster en conséquence l'exposant.
4. Arrondi: Le résultat de l'addition peut nécessiter un arrondi pour tenir compte de la précision limitée de la représentation en virgule flottante.

Passons maintenant à l'étude des différentes configurations de mapping et le temps de réponse.

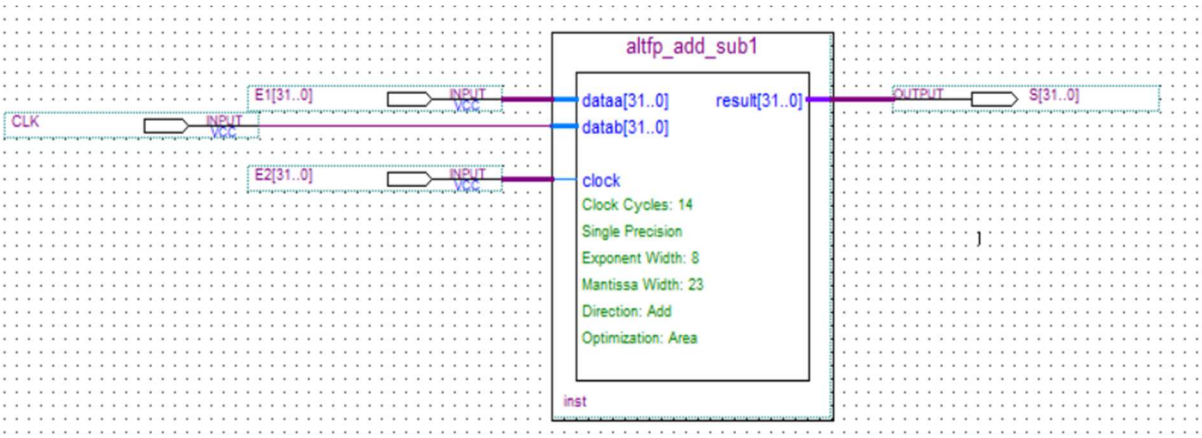


Figure 3 : additionneur flottant _ area

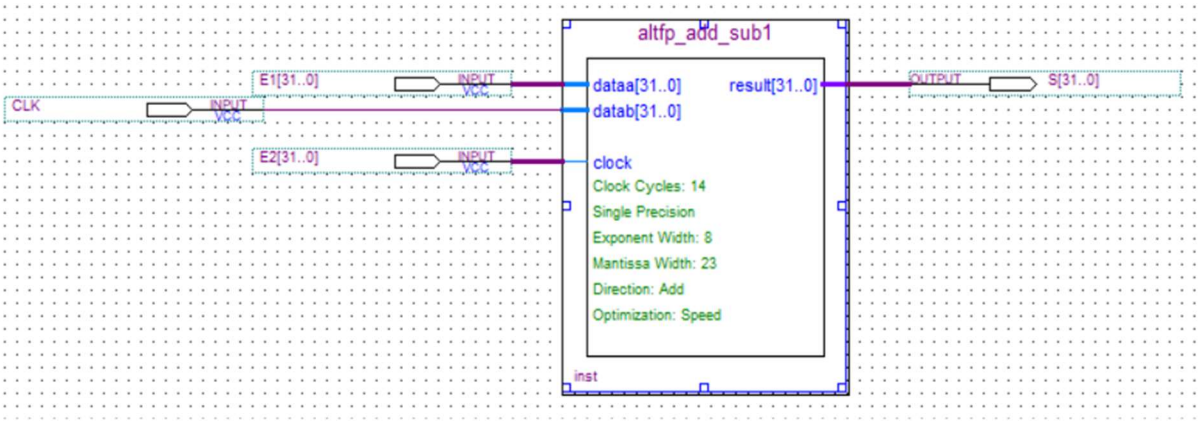


Figure 4 : additionneur flottant _ speed

Tableau d'étude des différents temps de réponse

	Area	Speed
Logique élément (LE)	962/15.408	1,096/15.408
Tsu	6,007 ns	6,520 ns
Tcom	7.658 ns	8,192 ns
Th	0,643 ns	0,694 ns
Σ(Tcom+Tsu)	13,665 ns	14, 712 ns

2.3 - On se propose d'étudier l'opérateur de la multiplication. Pour différentes implémentations avec bloc multiplieur dédié ou avec LUT, déterminer le temps de réponse et le pourcentage d'utilisation du FPGA



Figure 5 : l'opérateur de multiplication

Tableau d'étude des différents résultats

	Bloc multiplieur	LUT
Logique élément (LE)	79/15.408	1,205/15.408
Tpd	20,816 ns	31,638ns
Pourcentage	>1%	8%

2.4 – On souhaite réaliser un filtre vidéo :

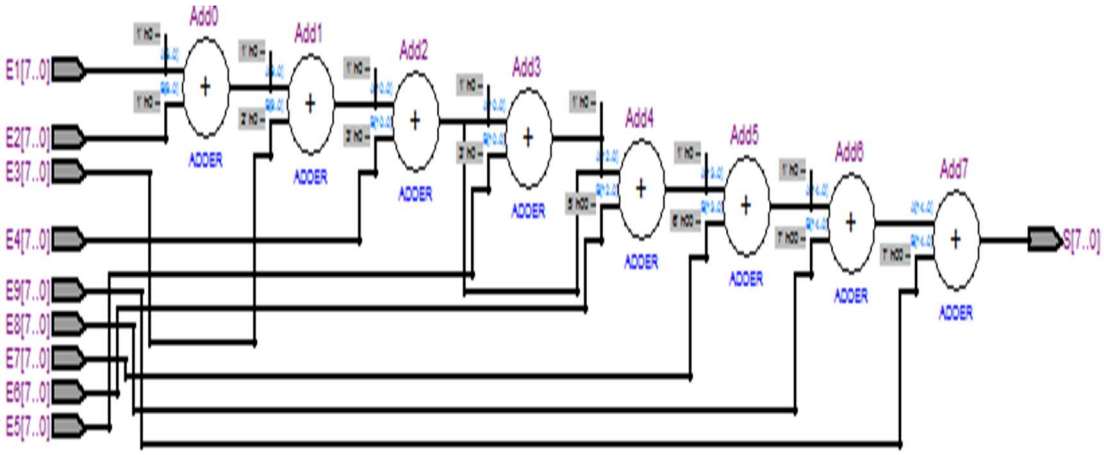


Figure 6 : Filtre Vidéo

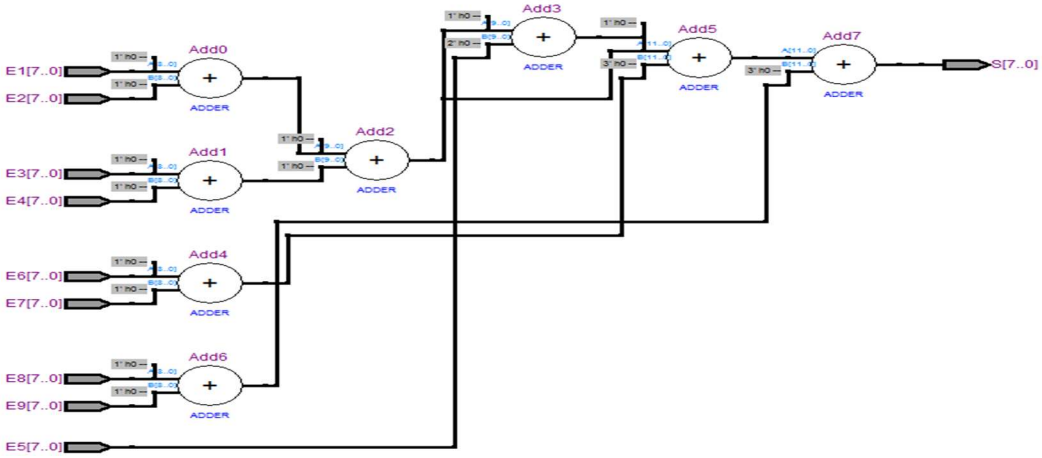


Figure 7 : Parallélisme filtre vidéo

Tableau d'étude des temps de propagation (Tpd)

	Tpd	LE
Filtre vidéo (sans parallélisme)	16,420 ns	63
Filtre vidéo (avec parallélisme) S<=(E1+E2+E3+E4)+(2*E5)+(E6+E7+E8+E9)	15,647 ns	63
Filtre vidéo (avec parallélisme) S<=(E1+E2)+(E3+E4)+(2*E5)+(E6+E7)+(E8+E9)	14,722 ns	63

COMMENTAIRE : Nous constatons dans ce tableau ci-dessus une réduction progressive du temps de propagation dû au parallélisme, on voit bien que sans parallélisme, le Tpd est de 16,420 ns, tandis qu'avec parallélisme, il descend à 15,647 ns puis à 14,722 ns. Cela démontre que le parallélisme permet d'accélérer le traitement en exécutant certaines opérations en parallèle . Donc nous pouvons conclure que l'utilisation du parallélisme dans le traitement du filtre vidéo permet de gagner en vitesse d'exécution.

CONCLUSION

Ce TP nous a permis de connaitre les concepts de programmation sur un FPGA en Cyclone III, de réaliser l'architecture des cellules élémentaires et leur optimisation, notamment en analysant l'impact des opérateurs arithmétiques tels que l'addition et la multiplication sur le nombre de cellules logiques (LE) utilisées et le temps de réponse (Tpd), à étudier des configurations avec ou sans pipeline, et à observer les gains en performances grâce à des outils comme le Classic Timing Analyzer de Quartus.

Ensuite nous avons porté une attention particulière à la synthèse matérielle et au mapping avec des études sur des opérateurs travaillant sur des entiers et des flottants, et la dernière partie du TP nous a permis d'explorer l'implémentation d'un filtre vidéo en FPGA avec une analyse des avantages du parallélisme sur la rapidité d'exécution. L'ensemble du travail démontre comment les décisions architecturales influencent l'efficacité des systèmes sur FPGA, notamment en termes d'optimisation des ressources et de réduction des temps de traitement.

Ce TP a été réalisé en trio, ce qui a permis une complémentarité des compétences et une répartition efficace des tâches, travailler à trois nous a permis d'avoir des perspectives différentes sur les problèmes rencontrés, ce qui a enrichi nos réflexions.

Nous tenons également à remercier les efforts de notre chargé de TP Mme Belleudy qui nous a aider à surmonter les difficultés rencontrées durant ce TP.

CONCLUSION

Ce TP nous a permis de connaitre les concepts de programmation sur un FPGA en Cyclone III, de réaliser l'architecture des cellules élémentaires et leur optimisation, notamment en analysant l'impact des opérateurs arithmétiques tels que l'addition et la multiplication sur le nombre de cellules logiques (LE) utilisées et le temps de réponse (Tpd), à étudier des configurations avec ou sans pipeline, et à observer les gains en performances grâce à des outils comme le Classic Timing Analyzer de Quartus.

Ensuite nous avons porté une attention particulière à la synthèse matérielle et au mapping avec des études sur des opérateurs travaillant sur des entiers et des flottants, et la dernière partie du TP nous a permis d'explorer l'implémentation d'un filtre vidéo en FPGA avec une analyse des avantages du parallélisme sur la rapidité d'exécution. L'ensemble du travail démontre comment les décisions architecturales influencent l'efficacité des systèmes sur FPGA, notamment en termes d'optimisation des ressources et de réduction des temps de traitement.

Ce TP a été réalisé en trio, ce qui a permis une complémentarité des compétences et une répartition efficace des tâches, travailler à trois nous a permis d'avoir des perspectives différentes sur les problèmes rencontrés, ce qui a enrichi nos réflexions.

Nous tenons également à remercier les efforts de notre chargé de TP Mme Belleudy qui nous a aider à surmonter les difficultés rencontrées durant ce TP.